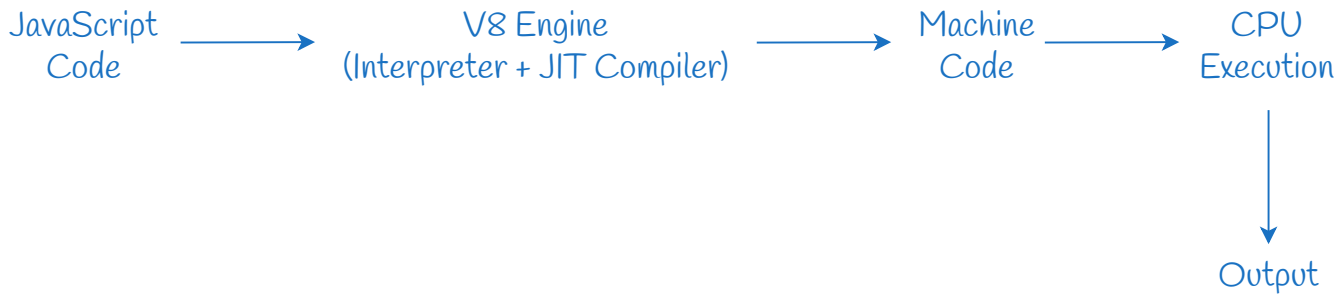


Lecture 02: The V8 Engine, Node.js, and JavaScript Fundamentals

How JavaScript Code Actually Runs?



What is the V8 Engine?

The V8 Engine is Google's open-source, high-performance JavaScript engine. It is a program entirely written in C++ and primarily designed to execute JavaScript code. Its primary job is to take human-readable JavaScript code and compile it directly into native machine code so the computer's processor can execute it.

Note:

- CPUs can only understand **Machine Code** (binary 0s and 1s).
- Machine Code is **hardware-dependent**, i.e., machine-dependent.

The C++ Connection: JavaScript itself is just a set of text rules (a specification called ECMAScript). When people say "JavaScript is written in C++", they actually mean the **engine**(V8) that reads JavaScript written in C++. It reads the JS text, parses it and compiles it into the specific machine code.

Computer Science Trivia: How was the first C++ compiler made before C++ existed?

This is a famous concept called **Bootstrapping**.

Phase 1: Pure Machine Code (The 1940s)

Early programmers had no software. They wrote code entirely in raw 0s and 1s, feeding instructions directly into the CPU using physical switches or paper punch cards.

Phase 2: Assembly Language

To escape tedious binary, programmers used raw 0s and 1s to build a tiny program called an **Assembler**. This allowed them to use short English abbreviations (like **ADD**, **MOV**, **SUB**).

Phase 3: The First High-Level Compilers

Using Assembly language, developers wrote the very first primitive compilers for early, human-readable languages like "B" and eventually "C".

Phase 4: Writing C++

Once they had a fully functional "C" compiler, Bjarne Stroustrup used the "C" language to write the very first "C++" compiler.

Phase 5: Self-Hosting

Once that first C++ compiler worked, they didn't need "C" anymore! They immediately used that working compiler to write the *next, better* version of C++ completely in C++. When a language can compile itself, it is called **Self-Hosting**.

How to Run JavaScript Outside the Browser?

JavaScript can run outside the browser using Node.js.

What is Node.js?

Node.js is an open-source, cross-platform Runtime Environment that allows JavaScript to execute outside the browser.

Node.js → **Google V8 Engine** (to execute the JS) + **libuv** (A C++ library to handle asynchronous tasks like the File System, Network, and OS interactions) + **Core APIs**.

How to Install Node.js on Your System?

1. Download Node.js from the official Node.js website.
2. Select your operating system (Windows/MacOS/Linux).
3. Run the installer and finish the installation using default settings.
4. Verify the installation by typing `node -v` in Command Prompt or Terminal.

Important Note: Do not apply the rules of other programming languages here. JavaScript handles memory, types, and variables in its own unique way.

Variables:

In modern JavaScript, variables are declared using **let** and **const**.

```
let a = 10;  
const b = 20;
```

The Difference Between *let* and *const*:

let

- Declares a variable whose value can change later.
- Can be declared without a value.

const

- Declares a variable whose value is *constant* and cannot change.
- Must be initialized with a value.

Note: JavaScript is a **dynamically typed** language. The JS Engine automatically detects the data type at runtime based on the value.

Data Types:

Data types describe the nature of data being stored.

1. Primitive Data Types

- **Number:** Represents both integers and decimal numbers.
- **String:** Represents textual data.
- **Boolean:** Represents a logical entity and can only have two values: *true* or *false*.
- **Undefined:** A variable that has been created but not yet assigned a value.
- **Null:** A special value that explicitly represents "nothing" or an "empty" value.
- **BigInt:** Used to store incredibly large numbers that standard *Number* cannot handle.
- **Symbol:** Used to create completely unique, hidden identifiers.

2. Non-Primitive Data Types

- **Object:** A collection of related data stored in key-value pairs.
- **Array:** An ordered list of multiple values stored in a single variable.
- **Function:** A reusable block of code that can be stored in a variable and executed later.

Important Note: `console.log()` is not a part of the core JavaScript language (ECMAScript). The **console** object is an external tool provided to JavaScript by the host **Runtime Environment** (such as the web browser or Node.js).

The typeof Operator:

The **typeof** operator is a built-in JavaScript tool used to determine the exact data type of a specific value or variable at runtime.

The null Bug: This is an infamous historical bug from the very first version of JavaScript created in 1995. **null** is fundamentally a Primitive data type, but **typeof** incorrectly reports it as an "object". The ECMAScript committee refuses to fix this bug today because doing so would crash millions of older websites that rely on this broken behavior.